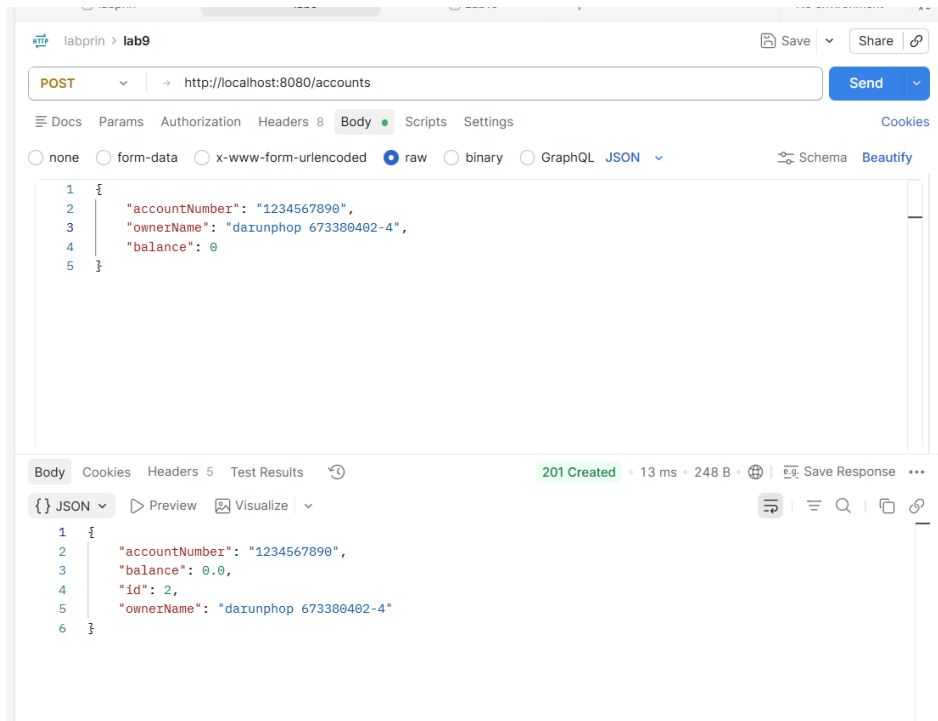


1. การสร้าง Account

- รูปภาพ

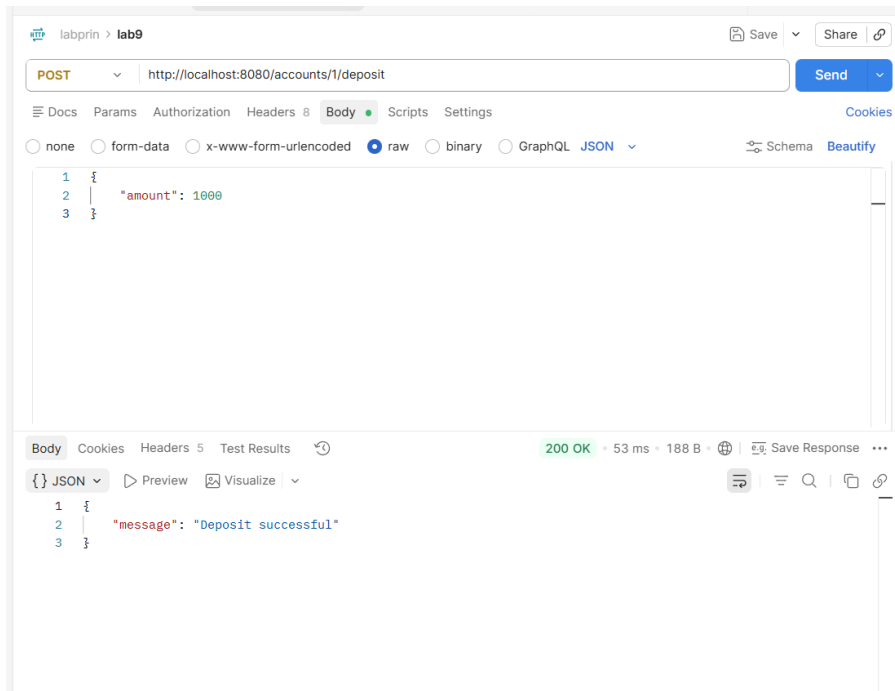


- อธิบายผล

Request นี้ทำงานสำเร็จ — ระบบได้สร้างบัญชีใหม่ให้กับ darunphop 673380402-4 ที่ localhost พร้อมยอดเงินเริ่มต้น 0 บาท และได้รับ ID = 2 กลับมา

2. การฝากเงินสำเร็จ

- รูปภาพ

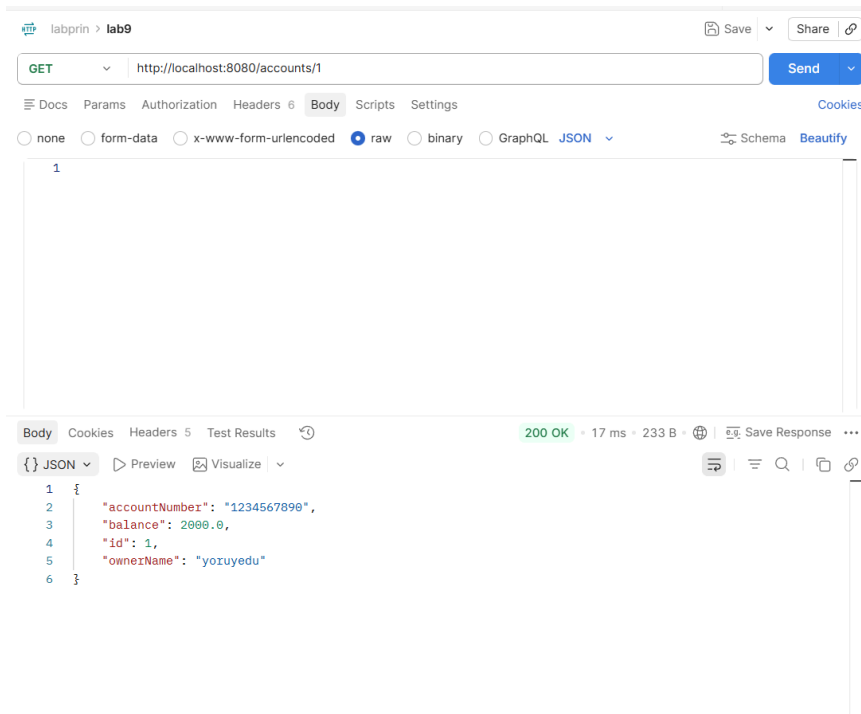


- อธิบายผล

ทดสอบ API ฝากเงินจำนวน 1,000 บาทเข้าบัญชี ID 1 ผลการทดสอบสำเร็จ โดยได้รับสถานะ 200 OK และข้อความ “Deposit successful”

3. ตรวจสอบ Account

- รูปภาพ



- อธิบายผล

ทดสอบ API สำหรับเรียกดูข้อมูลบัญชี ID 1 ผลการทดสอบสำเร็จ โดยได้รับสถานะ 200 OK และแสดงข้อมูลเลขบัญชี ยอดเงินคงเหลือ 2,000 บาท

4. ตรวจสอบ DepositTransaction

- รูปภาพ

	id [PK] bigint	amount double precision	account_id bigint
1	4	1000	1
2	5	1000	1

- อธิบายผล

รายการฝากเงินของบัญชี ID 1 โดยมีรายการฝากเงิน 2 รายการ รายการละ 1,000 บาท รวมเป็นเงิน 2,000 บาท

5. ทดลอง Rollback

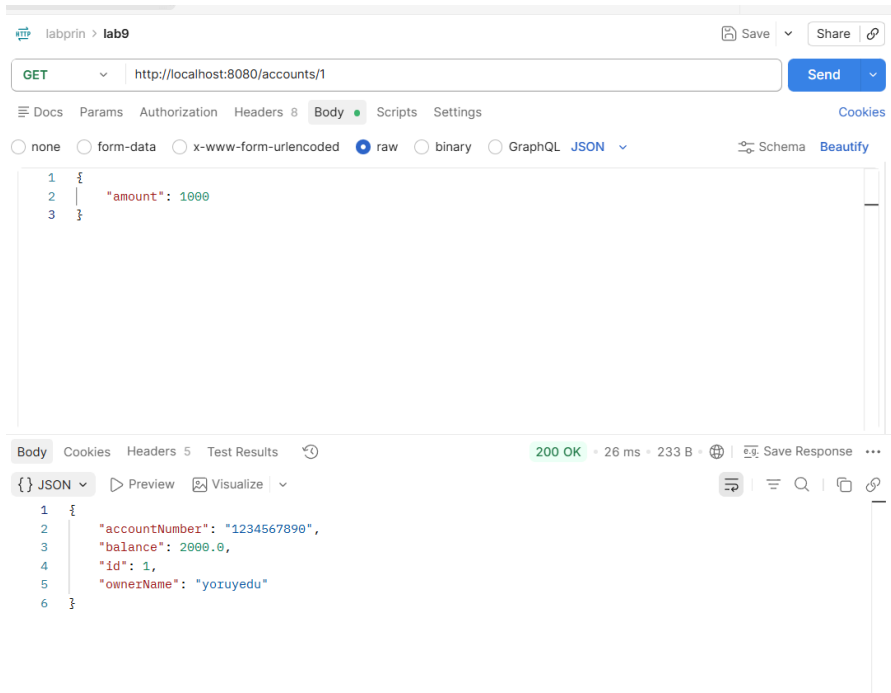
- รูปภาพ

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/accounts/1/deposit`. The request body is a JSON object: `{ "amount": 1000 }`. The response is a `500 Internal Server Error` with a status of 500. The error message is "Internal Server Error". The stack trace is as follows:

```

{
  "timestamp": "2026-09-11T07:50:24.861Z",
  "status": 500,
  "error": "Internal Server Error",
  "trace": "java.lang.RuntimeException: Test Rollback\r\n\tat com.example.lab9.service.DepositService.deposit(DepositService.java:37)\r\n\tat java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(Unknown Source)\r\n\tat java.base/java.lang.reflect.Method.invoke(Unknown Source)\r\n\tat org.springframework.aop.support.AopUtils.invokeJoinpointUsingReflection(AopUtils.java:359)\r\n\tat org.springframework.aop.framework.ReflectiveMethodInvocation.invokeJoinpoint(ReflectiveMethodInvocation.java:190)\r\n\tat org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvocation.java:158)\r\n\tat org.springframework.transaction.interceptor.TransactionInterceptor$1.proceedWithInvocation(TransactionInterceptor.java:133)\r\n\tat org.springframework.transaction.interceptor.TransactionAspectSupport."

```



	id [PK] bigint	amount double precision	account_id bigint
1	4	1000	1
2	5	1000	1

- อธิบายผล

ส่งคำขอฝากเงิน **1,000 บาท** ผ่าน Postman ระบบเกิด **500 Internal Server Error** ตามที่คาดไว้
เนื่องจากการเรียก RuntimeException("Test Rollback")

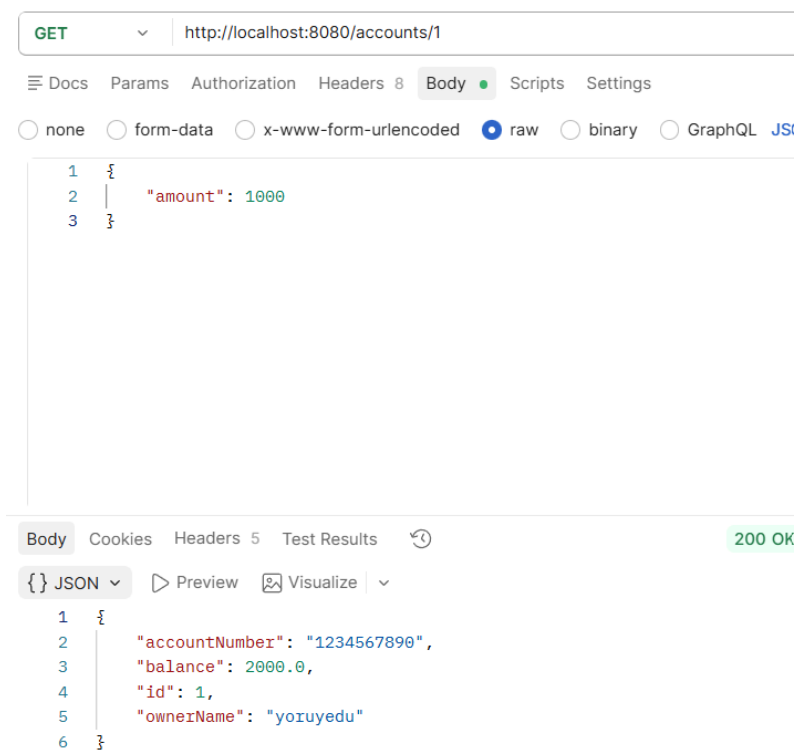
ตรวจสอบยอดเงินของบัญชี ID 1 พบว่ายอดเงินยังคงเป็น **2,000 บาท** แสดงว่าการเพิ่มเงิน 1,000 บาท
ถูก **Rollback** และไม่มีการเปลี่ยนแปลงยอดเงิน

ตรวจสอบตาราง deposit_transaction พบว่ามีเพียงรายการเดิม **2 รายการ** รายการละ **1,000 บาท**
และไม่มีรายการใหม่เพิ่มเข้ามา แสดงว่า Transaction ถูก **Rollback** สำเร็จ

6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional

- รูปภาพ

ตอนมี @Transactional



ตอนไม่มี @Transactional

POST http://localhost:8080/accounts/1/deposit

Docs Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   |   "amount": 1000
3 }
```

Body Cookies Headers 4 Test Results 500 Internal S

{ } JSON Preview Debug with AI

```
1 {
2   |   "timestamp": "2026-09-11T08:03:05.880Z",
3   |   "status": 500,
4   |   "error": "Internal Server Error",
5   |   "trace": "java.lang.RuntimeException: Test Rollback\r\n\tat com.example
   |           deposit(DepositService.java:37)\r\n\tat com.example.lab9.controller
```

GET http://localhost:8080/accounts/1

Docs Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   |   "amount": 1000
3 }
```

Body Cookies Headers 5 Test Results 200 OK • 11 m

{ } JSON Preview Visualize

```
1 {
2   |   "accountNumber": "1234567890",
3   |   "balance": 3000.0,
4   |   "id": 1,
5   |   "ownerName": "yoyuyedu"
6 }
```

	id [PK] bigint	amount double precision	account_id bigint
1	4	1000	1
2	5	1000	1
3	7	1000	1

- อธิบายผล

มี @Transactional → เกิด Error → ระบบทำ ROLLBACK → ข้อมูลทั้งหมดถูกย้อนกลับ → Database กลับไปเหมือนเดิม

ไม่มี @Transactional → เกิด Error → ไม่มีการ Rollback → ข้อมูลที่บันทึกไปก่อนเกิด Error ยังคงอยู่ใน Database

@Transactional ช่วยให้การทำงานหลายขั้นตอนเป็นชุดเดียวกัน หากเกิดข้อผิดพลาด ระบบจะยกเลิกการเปลี่ยนแปลงทั้งหมดเพื่อป้องกันข้อมูลไม่ตรงกัน

7. ตอบคำถามท้าย Lab ทั้ง 6 ข้อ

1. @Transactional มีหน้าที่อะไร?

ควบคุมการทำงานของ Transaction หากเกิด Error ระบบจะ **ROLLBACK** ข้อมูลกลับไปก่อนเริ่ม Transaction

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เพื่อให้การเพิ่มยอดเงินและบันทึกรายการฝากเงิน **สำเร็จพร้อมกัน** หรือยกเลิกพร้อมกัน ป้องกันข้อมูลไม่ตรงกัน

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT = ยืนยันและบันทึกข้อมูลลง Database

ROLLBACK = ยกเลิกการเปลี่ยนแปลงและคืนข้อมูลกลับเหมือนเดิม

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

เป็นความสัมพันธ์แบบ **One-to-Many (1:N)** คือ 1 บัญชีสามารถมีรายการฝากเงินได้หลายรายการ

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ข้อมูลที่เปลี่ยนแปลงภายใน Transaction จะถูก **ROLLBACK** กลับไปเหมือนเดิม

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

เมื่อเกิด Error ข้อมูลที่บันทึกไปก่อนเกิด Error **ยังคงอยู่ใน Database** เพราะไม่มีการ Rollback อัตโนมัติ